

Foreign Key Actions

When a foreign key column in **R** references **S**, a change to **S** (delete or update) may leave rows of **R** pointing at nothing. You choose what should happen.

Action	What it does on the child side
NO ACTION / RESTRICT	Reject the delete/update on the parent. <i>Default.</i>
CASCADE	Repeat the change on the child: delete the child row, or update the child's FK to the new value.
SET NULL	Leave the child row, but set its FK column to NULL.

You can pick ON DELETE and ON UPDATE actions *independently*:

```
CREATE TABLE Sells (
  bar CHAR(20), beer CHAR(20), price REAL,
  FOREIGN KEY (beer) REFERENCES Beers(name)
    ON DELETE SET NULL ON UPDATE CASCADE
);
```

Inserts into the child that reference a missing parent are **always** rejected — these actions only control what happens on the parent side.

SELECT-FROM-WHERE

```
SELECT <attributes or expressions>    -- what to return
FROM   <one or more tables>           -- where to read from
WHERE  <condition on rows>;           -- which rows to keep
```

* all columns of the (single) FROM table
 col AS newname rename column in the result (table is untouched)
 expr AS newname compute a value per row, e.g. tuition*1.13
 'some text' AS a constant column, same value on every row
 label
 t.col qualify with a table alias from FROM Courses t

Our Schema (used on Side B)

Underlines mark key attributes.

Students(sid, name, email) Courses(cid, title, dept) Instructors(name, office, phone)
 Likes(student, course) Teaches(instructor, course, semester) Enrolled(student, course, tuition)

Students

sid	name	email
s001	Ava	ava@u.ca
s002	Ben	ben@u.ca
s003	Cora	NULL
s004	Dev	dev@u.ca

Likes

student	course
s001	CSC343
s001	CSC311
s002	CSC343
s003	MAT223
s004	CSC343

Courses

cid	title	dept
CSC343	Databases	Comp Sci
CSC311	ML	Comp Sci
CSC207	Software Design	Comp Sci
MAT223	Linear Alg. I	Math
PSY100	Intro Psych.	Psych

Enrolled

student	course	tuition
s001	CSC343	800
s002	CSC311	800
s003	MAT223	750
s004	CSC207	800
s004	PSY100	750

Instructors

name	office	phone
Sibia	DH3074	1234
Liu	DH2021	5678
Park	DH3000	NULL

Teaches

instructor	course	semester
Sibia	CSC343	Summer26
Liu	CSC311	Summer26
Park	MAT223	Summer26

Use Side A's schema and sample data. Write actual SQL, not pseudocode. Boxes marked (*discuss*) are debriefed with the class after individual work.

Box 1 Pick the FK Action (4 min, discuss)

For each situation, choose one: CASCADE / SET NULL / RESTRICT. Write a one-line reason.

Situation	Action	Reason
A Course is deleted. We should drop every Enrolled row for it.		
A cid like CSC343 is renamed to CSC443 . The change should follow into Likes .		
An Instructor resigns. Teaches rows should stay as a record, but no longer name them.		
A Student is being deleted, but they still have Enrolled rows. We don't want to lose those.		

Box 2 Read the Query (4 min)

Using the sample data on Side A, write the result of each query.

```
SELECT title
FROM Courses
WHERE dept = 'Comp Sci';
```

```
SELECT student, tuition * 1.13 AS withTax
FROM Enrolled
WHERE course = 'CSC343';
```

Box 3 Write From Scratch (8 min)

Write SQL queries against the Side A schema.

(a) The titles of all courses in the **Math** department.

(b) Every student who likes **CSC343**, returned in a column called **fan**.

(c) Every row of **Enrolled**, with the tuition expressed in cents (column name: **tuition_cents**).

Box 4 Spot the Errors (4 min, discuss)

The query below has **three mistakes**. Circle each and write a fix.

```
SELECT  sid, name 'is a student' AS label
FROM    Student
WHERE   sid = s001;
```

- (a) *fix:*
- (b) *fix:*
- (c) *fix:*